

Process × File Reference

文件描述符不是文件：三级解耦架构怎样决定 fork、dup、close 与 unlink

408 · OS-B03

Mother Interface

Task / Process

→ descriptor binding → Open Instance / OFD

→ object reference → File Object

→ FS mapping → Persistent Data / Metadata

本 Bridge 专门负责厘清“进程侧句柄绑定、系统打开实例、文件对象”之间的引用交接契约；路径解析、文件对象生命周期的最终回收协议与块分配均由 OS-06/07 独立拥有。

先定义接口两侧的词

- **文件描述符 (fd)**：进程可见的整数句柄，用来选择本进程描述符表中的一个绑定。它不是文件身份，也不直接等于 inode。
- **Descriptor Table / Binding (描述符表/绑定)**：进程级状态，记录某个 fd 当前引用哪个打开实例，以及少量 descriptor-local 属性。具体是数组、分层表还是其他结构属于实现。
- **Open File Description / Open Instance (OFD/打开实例)**：一次“打开”产生的系统级运行状态，典型地拥有当前文件位置、打开状态和对底层文件对象的引用。多个 fd 可以共享同一个打开实例。
- **File Object (文件对象)**：由文件系统标识和维护的持久对象；经典 UNIX 可用 inode 理解其身份与元数据。身份只在相应文件系统实例/命名空间契约下解释；**文件名属于命名关系，不是 inode 本体的一部分。**

Owners 与职责边界

- **左侧 Owner (OS-01/02 进程控制)**：拥有进程生命周期，以及 fork()/exec()/exit() 对进程资源引用的高层交接语义。
- **右侧 Owner (OS-06/07 文件系统)**：拥有 pathname、打开实例、文件对象、命名引用、数据块映射与最终回收/持久化协议。
- **本 Bridge Owns**：进程描述符绑定怎样跨 fork/dup/close 与打开实例交接，以及“共享同一打开实例”怎样推出共享当前文件位置等运行状态。

1 核心机制：引用的三级解耦数据结构

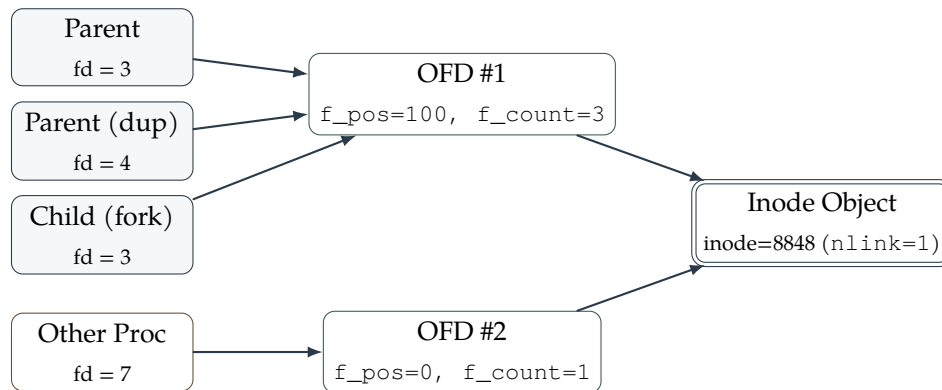
稳定模型不是记 Linux 结构体名字，而是保留三层独立身份：

1. **进程句柄层**：当前进程用 fd 选择一个描述符绑定；不同进程可以使用相同整数 fd 指向完全不同的对象；
2. **打开实例层**：一次打开产生运行时状态。若多个描述符绑定引用同一 OFD，它们共享该 OFD 所拥有的当前文件位置等状态；

3. **文件对象层**：多个独立 OFD 仍可以引用同一个文件对象，但各自保持独立的打开实例状态。

这三层使“共享文件对象”与“共享当前 offset”成为两个不同问题。

2 母模型：独立打开、`fork()` 继承与 `dup()` 拓扑



2.1 读写位置 (`f_pos`) 共享判定黄金法则

- 共享同一个 OFD (游标共同推进)：

- `fork()`：子进程复制父进程的 `fd_table`，父子相同 `fd` 指向同一 OFD，父进程读写会影响子进程的后续读取位置；
- `dup(oldfd)` / `dup2()`：同一进程产生新 `fd`，指向同一 OFD，两个 `fd` 操作同一个读写游标。

- 拥有各自独立的 OFD (游标互不影响)：

- 两个进程 (或同一进程两次) 独立调用 `open("same.txt", ...)`，各自生成独立 OFD，拥有独立的 `f_pos`，但最终指向同一个 Inode。

3 常见操作怎样改变三层关系

操作 API	进程描述符绑定	打开实例 / OFD	文件对象 / 命名关系
<code>open()</code>	建立一个新的 fd 绑定	通常建立一个新的打开实例， <code>pathname</code> 被解析到一个文件拥有自己的 <code>current offset</code> 等状态	文件对象；创建/截断等效果按具体 <code>flags</code> 与 FS 契约处理
<code>fork()</code>	子进程得到描述符绑定的语义副本	父子对应绑定继续引用同一打开实例，因此共享其 <code>current offset</code>	不创造新的文件对象或硬链接
<code>dup/dup2()</code>	在同一进程建立另一个 fd 绑定	新旧 fd 引用同一打开实例	文件对象不因 <code>dup</code> 被复制
<code>close(fd)</code>	删除当前进程的一条 fd 绑定	该打开实例少一个引用；是否最终销毁由剩余引用决定	不等于删除命名关系；文件对象回收由 OS-06/07 生命周期协议决定
<code>link()</code>	与既有 fd 绑定无直接关系	不要求创建新打开实例	增加一个指向同一文件对象的命名引用
<code>unlink()</code>	既有 fd 绑定仍可继续存在	既有打开实例仍可引用该对象	删除一条命名引用；对象何时最终可回收取决于命名引用与打开引用等生命周期条件

4 经典反直觉案例：unlink() 正在被打开的文件

若进程 P 已经打开 `/tmp/temp.dat`，此时另一执行路径执行 `unlink("/tmp/temp.dat")`：

1. 文件系统删除的是该 `pathname` 对文件对象的一条命名引用；若没有其他同名/硬链接，新 `pathname` `lookup` 将无法再由这条名字找到对象；
2. P 的 fd 绑定和打开实例并不会因为名字消失而自动失效；
3. 因此 P 仍可通过既有 fd 继续按该打开实例的权限与 `offset` 访问对象；
4. 只有当文件系统判断命名引用和打开引用等所有维持对象生命的关系都已消失后，该对象才进入最终删除/空间回收流程；具体延迟回收、日志提交和缓存写回由 OS-06/07 Own。

Anti-Bridge：三个必须阻断的错误认知

- `fd ≠ Inode`：fd 只是进程私有整数代号；Inode 才是持久文件对象。
- `close(fd) ≠ Delete File`：`close()` 仅仅解除了当前进程的一个引用句柄，文件本身依然完好保存在文件系统中。
- `unlink() ≠ Immediate Disk Reclamation`：删除 `pathname` 只改变命名关系；对象何时成为可回收对象，要继续检查是否还有其他命名引用、打开引用以及文件系统自身尚未完成的持久化/回收状态。

30 秒极速调用协议

做题遇到 “fork/dup/open 多次读写、unlink 空间释放” 时，严格按三层追踪：

fd 绑定到哪个打开实例？

是否共享同一 OFD/current offset

命名引用与打开引用分别怎样变化？